Life Cycle of An Activity

IS2205 - Mobile Application Design & Development

Intended Learning Outcomes

At the end of this section you will be able to :

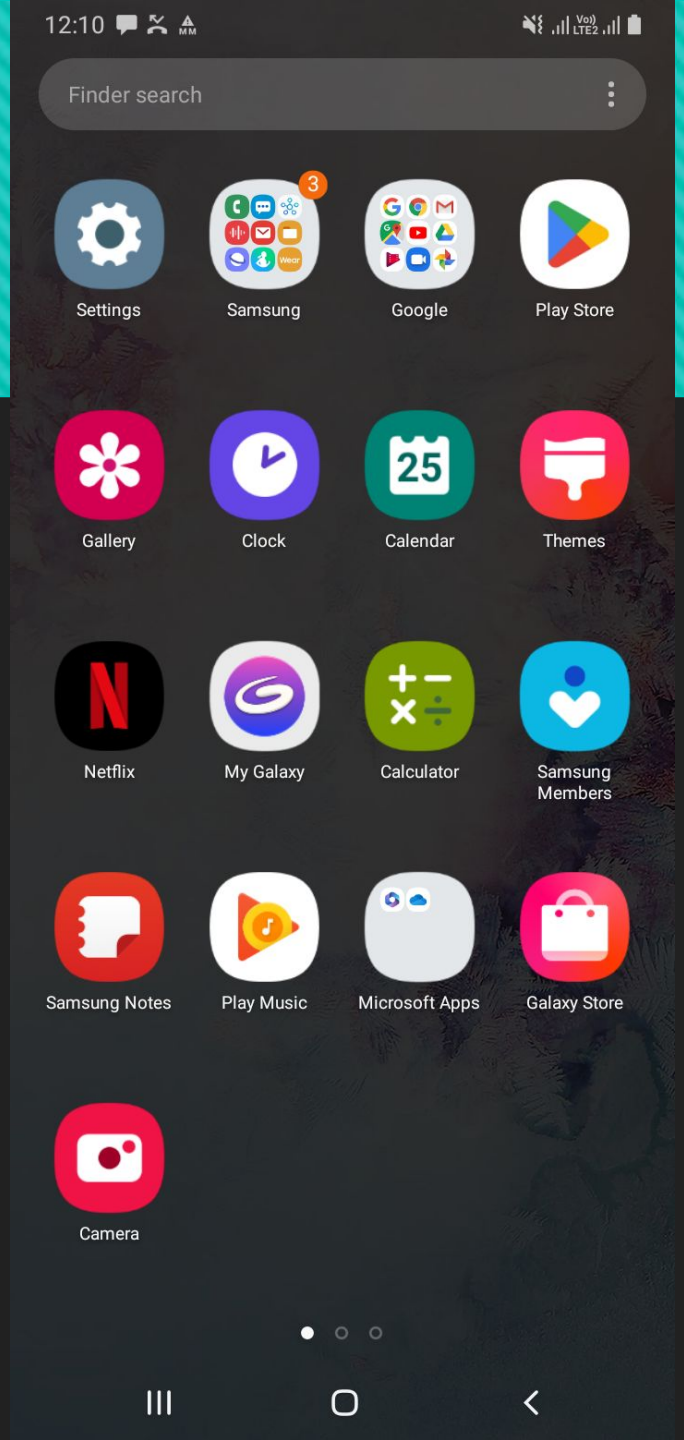
- ❑ Comfortably develop a minimal android app with a simple user interface
- ❑ Explain the design of an Android activity from a programming perspective
- ❑ Explain the life cycle of an Android activity
- ❑ Explain concepts of tasklist and backstack

Agenda

- App behavior
- Tasks & Back Stack
- Activities :
  - Lifecycle
  - Useful methods

Tasks

- The device Home screen is the starting place for most tasks. (User click)
- If no task exists for the app (the app has not been used recently)
- A new task is created and the "main" activity for that app opens as the root activity in the stack.



How does AOS identify the main?

- Usually applications have more than one activity
- Here we have one of the uses of AndroidManifest.xml file.
- A special definition is included to designate the primary activity of the App.
- Any other activities should be programmatically started

How does AOS identify the main?

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:label="@string/app_name"
    android:theme="@style/Theme.MyApp1">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

How does AOS identify the main?

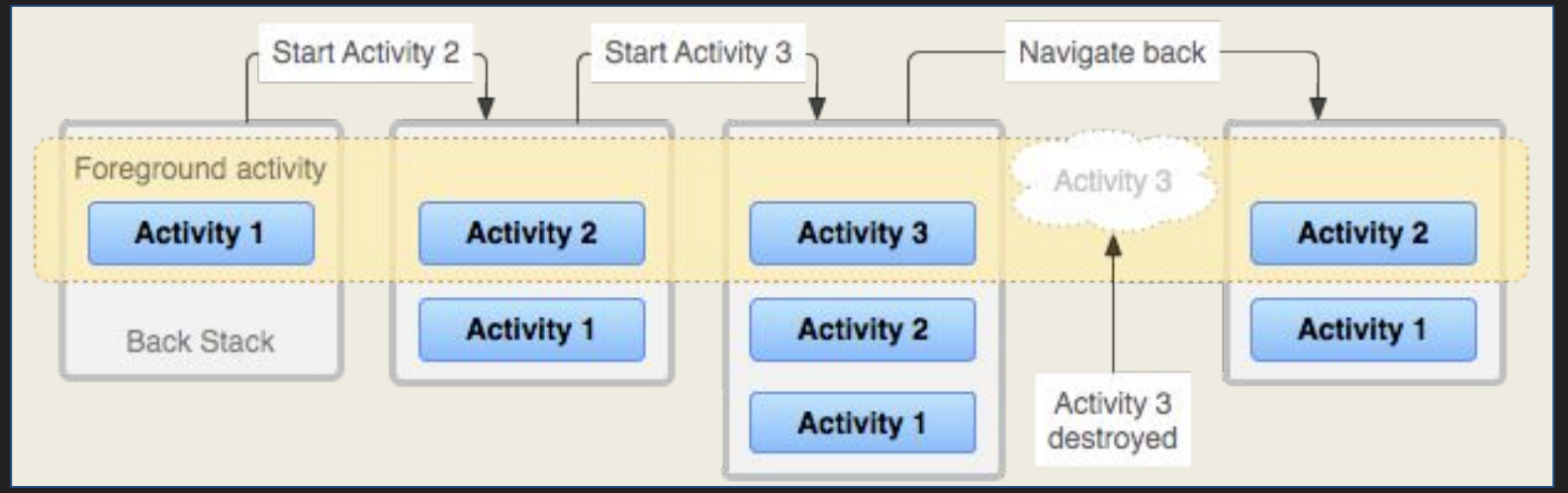
- Name – Name of the activity. Dot notation implies the app package
- Exported – Can the activity be launched by other components in the system or only by the internal components of the app?
- Label – Name of the application, externalized to a String file.
- Theme – theme reference for the UI
- Intent filter - <action android:name="android.intent.action.MAIN" /> - Indicates the main activity
  <category android:name="android.intent.category.LAUNCHER" /> - creates an icon in the launcher screen

Back Stack

- If current activity starts another, the new activity is pushed on the top of the stack and takes focus.
- The previous (top) activity remains in the stack, but is stopped.
- When the user presses the **Back** button, the current activity is popped from the top of the stack (the activity is destroyed) and the previous activity resumes (the previous state of its UI is restored).

Back Stack...

- **Activities in the stack are never rearranged**, only pushed and popped from the stack
- The back stack operates as a **"last in, first out"** object structure.
- Back Stack:



Back Stack and Tasks…

- A task is a cohesive unit that that groups activities
- When all activities are removed from the stack, the task no longer exists

Two tasks with activities:

Default behavior for activities and tasks : Summary

- Activity A starts Activity B; Activity A is stopped (the system retains its state) If Activity B is closed, Activity A resumes with restored state.
- By pressing the **Home** button if a task is abandoned, the current activity is stopped and its task goes to the background. The system retains the state of every activity in the task.
- On pressing **Back** button, the current activity is popped from the stack and destroyed. The previous activity in the stack is resumed.
- Activities can be instantiated multiple times, even from other tasks.

Activities

- The Activity class is a crucial component of an Android app, it is *close* to a **Window** in desktop platforms
- Activities are not launched from a main() method
- Android OS initiates code in an Activity instance by invoking specific **callback methods** that correspond to specific stages of its lifecycle
- Activities are loosely bound to a given app, and they may be launched from different apps

Question:

- What advantage(s) is(are) there when android activities are not tightly bound to an app?

Activities : Configuring the manifest

- To declare, an Activity, open manifest file and add an **<activity>** element as a child of the **<application>** element:
- Only required attribute for this element is **android:name**, i.e.: **class name** of the activity. Optional attributes that define activity characteristics includes label, icon, or UI theme.

```
<manifest ... >
  <application ... >
      <activity android:name=".ExampleActivity" />
      ...
  </application ... >
  ...
</manifest >
```

Activities : …manifest : declare intent filters

- Intent filters can be used to launch an Activity from a generic request, i.e.: allow user to select one of the email apps to handle an intent for drafting an email
- To declare an **<intent-filter>** attribute in the **<activity>** element. The definition includes an **<action>** element and, optionally, a **<category>** element and/or a **<data>** element. Together these specify the **type of intent** to which your activity can respond

```
<activity android:name=".ExampleActivity" android:icon="@drawable/ap
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="text/plain" />
    </intent-filter>
</activity>
```

Activities : …manifest : declare intent filters

- The **<action>** element specifies that this activity sends data. There are known actions for Android
- **<category>** is another parameter defined in Android to group intents
- The **<data>** element specifies the type of data that this activity can send.
- To restrict other apps from activating 'your' activities avoid declaring intent filters.

Code to launch an activity:

```
val intent = Intent(this, SecondActivity::class.java)
intent.putExtra("key", value)
startActivity(intent)
```

Activity on Actions

List 5 useful actions in the Android API

https://developer.android.com/reference/android/content/Intent#ACTION_AIRPLANE_MODE_CHANGED

Activities : Managing Lifecycle

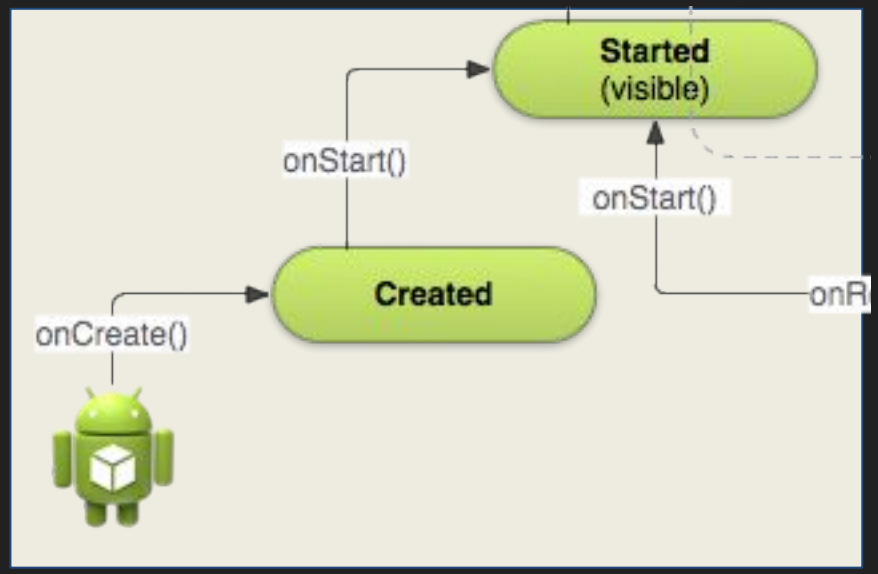
- Over the course of its lifetime, an activity goes through a number of states.
- A series of **callbacks** are used to handle transitions between states.
- We will look at these **callbacks**:

Activities : Managing Lifecycle : Callback?

- Callback method differs from a called method by the one who calls them
- Calculator object calls ⟶ multiply(a: int,b: int) method
- Call is initiated from the application end i.e.: user clicks a button, **programmer** has **coupled** click to the method
- **MainActivity** has **onCreate(b:** Bundle?) method which is called **by the Android OS** when activity is launched
- Programmer should **NOT** call a callback method from code

Activities : Managing Lifecycle : onCreate()

- IDE fills this callback by default, which fires when the system creates the activity.
- The implementation could initialize the essential components of the activity i.e.: The app should create views (Layout inflation etc....) and bind data to lists in here.

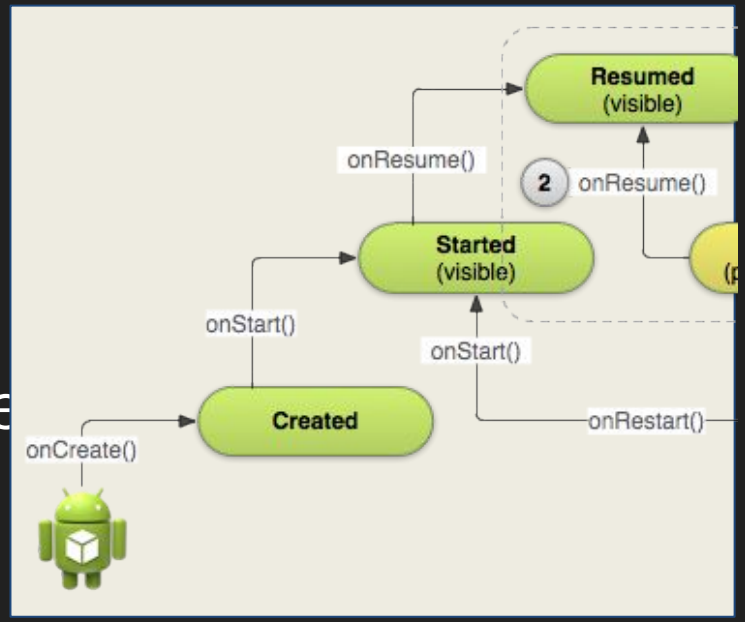


Activities : Managing Lifecycle : onCreate()

- Important! call **setContentView()** to define the layout for the activity's user interface.
- **onCreate**(savedInstanceState **Bundle?**) receives the parameter, which is a **Bundle** object containing the activity's previously saved state.
- When **onCreate()** finishes, the next callback is always **onStart()**.

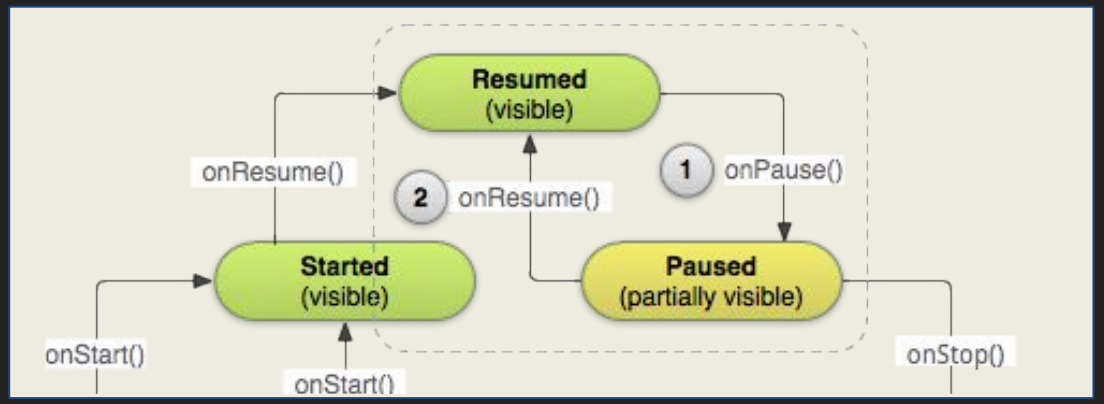
Activities : Managing Lifecycle : onStart()

- As onCreate() exits, the activity is in the **Created** state
- onStart() exits, the activity goes to the **Started** state
- The activity becomes visible to the user.
- This callback contains what amounts to the activity's final preparations for coming to the foreground and becoming interactive
- This method is the counterpart of onStop()



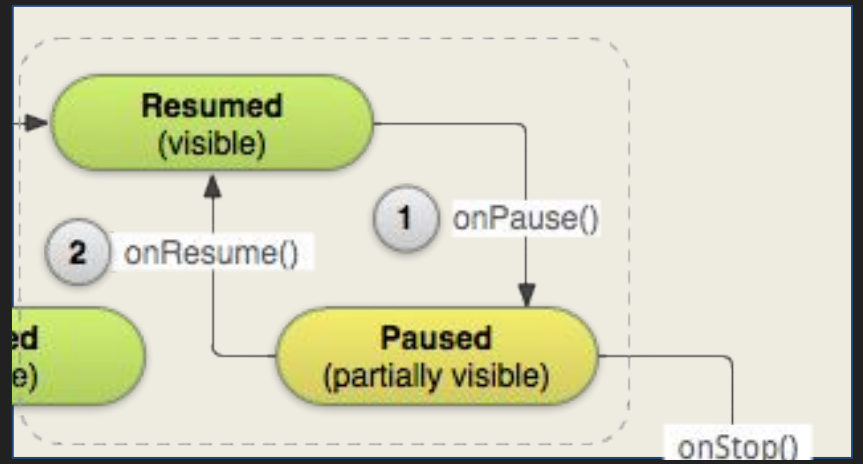
Activities : Managing Lifecycle : onResume()

- The system invokes this callback just before the activity starts interacting with the user
- At this point, the activity is at the top of the activity stack, and captures all user input
- Typically onResume() can be used to register with external events i.e.:Broadcasts
- The onPause() callback always follows onResume()



Activities : Managing Lifecycle : onPause()

- Android calls onPause() when the activity loses focus and enters a **Paused** state.
- This state occurs when, for example, the user taps the Back or even **Recents** button.
- If Android calls onPause(), *usually* it is an indication of user leaving the activity, unless it is resumed again.



Activities : Managing Lifecycle : onPause() …

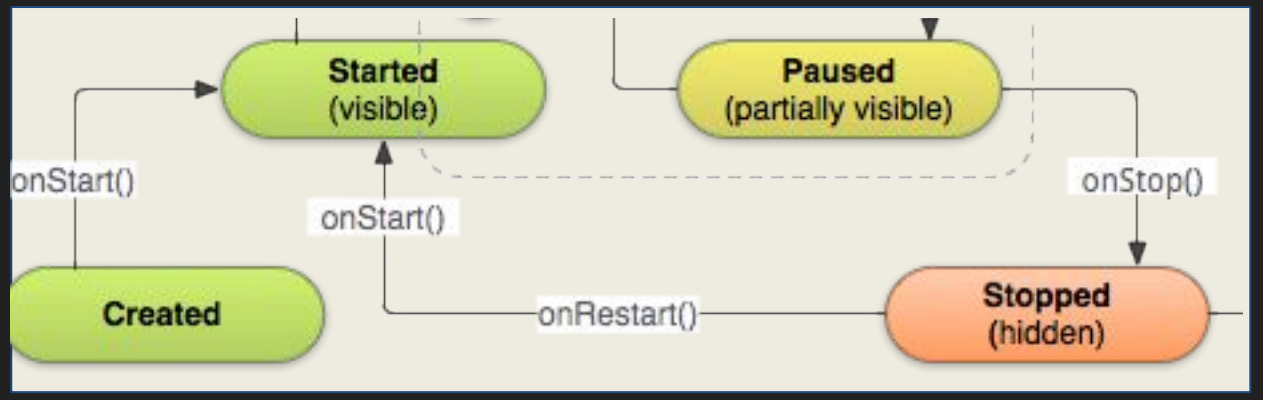
- An activity in the Paused state may continue to update the UI i.e.: a media player playing a song.
- Do not use onPause() to save application or user data, make network calls, or execute database transactions.
- These will take time and will display poor performance.
- Once onPause() returns, the next callback is either onStop() or onResume(), depending on what happens after the activity enters the Paused state.

Activities : Managing Lifecycle : onStop()

- Android calls onStop() when the activity is no longer visible to the user, Either due to:
  - This activity is being destroyed
  - A new activity is starting
  - An existing activity is entering a Resumed state while covering this activity
- After onStop() Android system calls:
  - onRestart() - if the activity is coming back to top
  - onDestroy() - if this activity is terminating completely

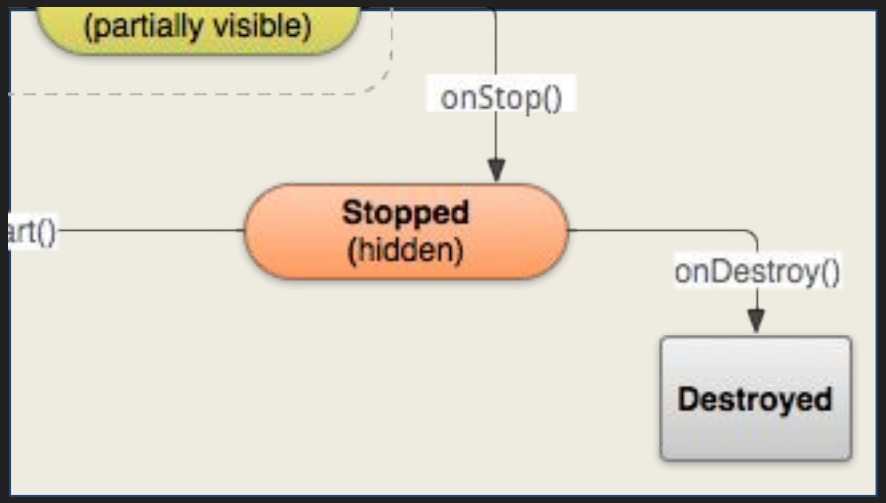
Activities : Managing Lifecycle : onRestart()

- The system invokes this callback when an activity in the **Stopped** state is about to restart.
- onRestart() restores the state of the activity from the time that it was stopped.
- This callback is always followed by onStart().



Activities : Managing Lifecycle : onDestroy()

- Android invokes onDestroy() before an activity is destroyed.
- The final callback that the activity receives.
- onDestroy() is usually implemented to ensure that all of an activity's resources are released when the activity, or the process containing it, is destroyed.

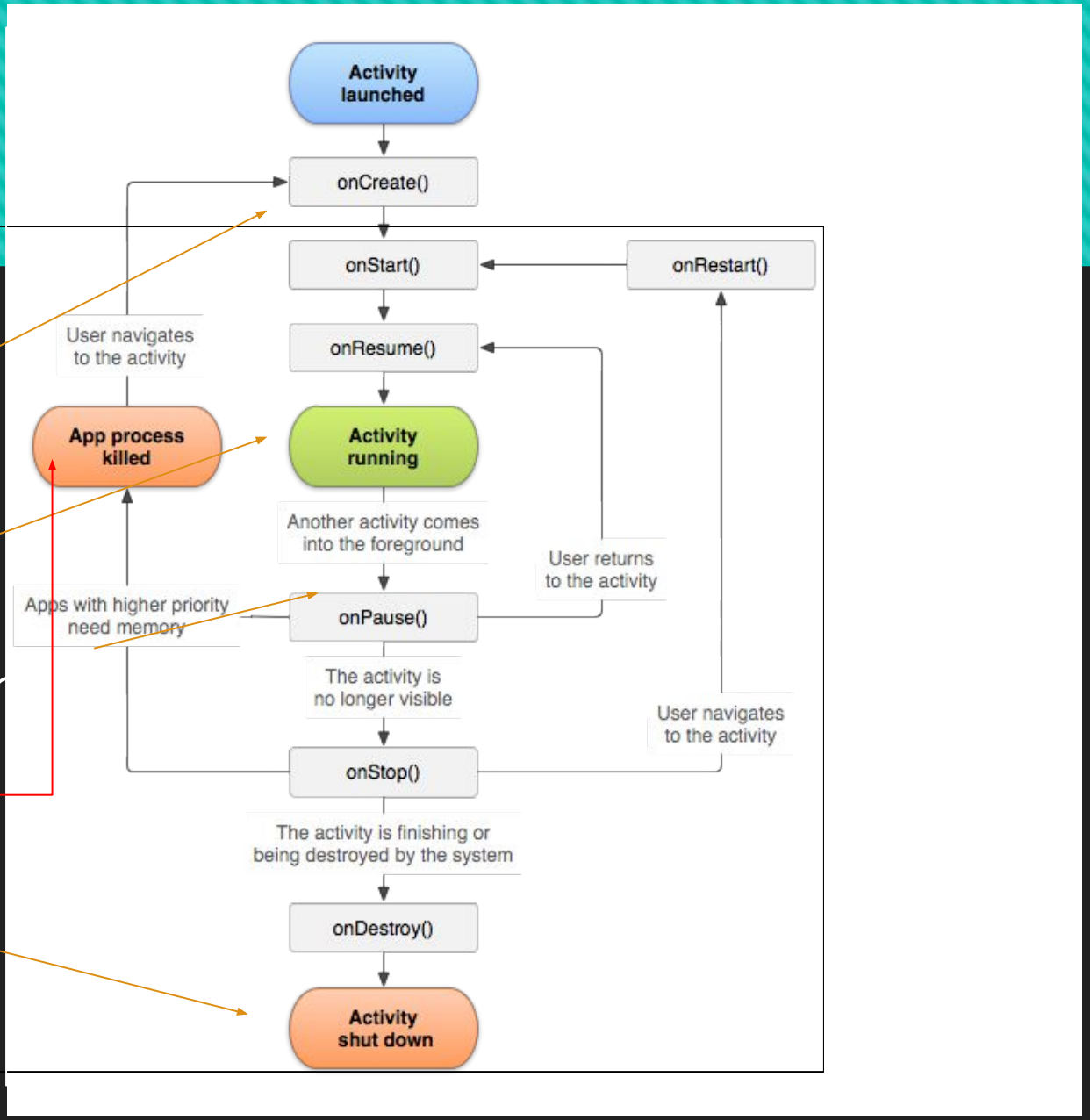


Activities : Managing Lifecycle : Multi-Window Mode

- Multi-window mode does not change the activity lifecycle.
- In multi-window mode, only the activity the user has most recently interacted with is active at a given time.
- This activity is considered topmost, and is the only activity in the **Resumed** state.
- All other visible activities are **Started** but are not **Resumed**.
- The system gives these visible-but-not-resumed activities higher priority than activities that are not visible.

Activities : Lifecycle

- Initializing
- User can interact
- Goes to the backgrour
- Force death
- Normal death



Activities : Managing Lifecycle : Summary

- The **entire lifetime** of an activity happens between the first call to **onCreate(Bundle)** through to the final call to **onDestroy**().
- The **visible lifetime** of an activity happens between a call to **onStart()** until a corresponding call to **onStop()**. The **onStart()** and **onStop()** methods can be called multiple times, as the activity becomes visible and hidden to the user.
- The **foreground lifetime** of an activity happens between a call to **onResume()** until a corresponding call to **onPause()**. During this time the activity is in visible, active and interacting with the user.

Activities : More Useful Methods : finish() & finishActivity(int)

- How to stop an Activity programmatically?
- Call **finish()** when the activity is done and should be closed.
- The ActivityResult is propagated back to whoever launched it via **onActivityResult()**.
- **finishActivity(requestCode :int)** - force finish another activity that this activity had previously started with **startActivityForResult()** call.
- Parameter requestCode - The request code given to **startActivityForResult()** when starting

Activities : More Useful Methods : getCallingActivity() & onBackPressed()

- **getCallingActivity()** Returns the name of the activity that invoked this activity.
- **onBackPressed()** is Called when the activity has detected the user's press of the **Back** key : This is deprecated now
- The default implementation simply finishes the current activity
- Can be overridden this to do ‘anything’; typical usage is getting confirmation to exit

Activities : More Useful Methods : onSaveInstanceState(Bundle)

- Called in order to save per-instance state from an activity before being ***killed*** so that the state can be restored in onCreate(Bundle) or onRestoreInstanceState(Bundle)
- The Bundle populated by this method will be passed to both methods.
- Implementation should populate the bundle with essential information
- Note that this saved Bundle only applies to force killed instances

Activities : More Useful Methods : onRestoreInstanceState(Bundle)

- This method is called after onStart() when the activity is being re-initialized from a previously saved state.
- Most implementations will simply use onCreate(Bundle) to restore their state, but it may be convenient/required to do it after all of the initialization has been done.
- Retrieve the information from the bundle and update the application data accordingly.
- Note that restoring only applies to force killed instances

Exercise

- Create a one activity application and include log messages for each callback method stating which callback is being called. Change the states by interacting with the activity. Observe the logs to identify the callback order.

References

https://developer.android.com/docs

https://developer.android.com/reference/android/app/Activity